

Applications of Binary Tree: Expression Tree & Huffman Encoding Data Structures

> **Definition: Binary Trees** are applied extensively in compilers (Expression Trees for arithmetic parsing) and data compression algorithms (**Huffman Encoding** for prefix-free lossless entropy coding).

1. Application 1: Expression Trees

An **Expression Tree** is a binary tree where **internal nodes** are operators (`+`, `-`, `*`, `/`) and **leaf nodes** are operands (`a`, `b`, `5`, `10`).

[*]

Expres

/ \

Traversals of Expression Tree:

1. **Inorder Traversal:** Yields **Infix Expression** with parentheses $\rightarrow ((a + b) * (c - d))$

2. **Pr**

Constructing Expression Tree from Postfix Expression:

1. Read postfix string token by token.

2. If to

2. Application 2: Huffman Encoding (Data Compression)

Huffman Coding is a **greedy algorithm** that assigns variable-length binary codes to characters based on their frequency of occurrence. More frequent characters get shorter codes.

Step-by-Step Huffman Tree Construction:

1. Count frequency of each unique character in input text.

2. Inse

[100]

Resulting Prefix-Free Codes:

- `A` -> `0` (1 bit)

3. Must-Write Points for Exams

- Huffman codes are **Prefix-Free Codes** (no character code is a prefix of any other code), enabling unambiguous decompression.

4. Quick Recall Flow
